

РЕФЕРАТ

Расчетно-пояснительная записка 42 с., 9 рис., 0 табл., 6 источн., 1 прил.

В работе представлены теоретические сведения о протоколе TCP, обработке сетевых соединений сетевой подсистемой Linux, DDoS атаке SYN Flood и методе борьбы с ним на основе TCP Cookies. Спроектированы схемы алгоритмов модуля ядра, реализующего метод защиты от SYN Flood атаки. Описано реализованное ПО и приведены примеры его работы.

Ключевые слова: DDoS, Linux, Сетевая подсистема, XDP, TCP, SYN Flood, TCP Cookies.

СОДЕРЖАНИЕ

РЕФЕРАТ	4
ВВЕДЕНИЕ	7
1 Аналитический раздел	8
1.1 Постановка задачи	8
1.2 Протокол TCP	8
1.2.1 Заголовок TCP сегмента	8
1.2.2 Трехстороннее рукопожатие	10
1.3 Обработка SYN запроса сетевой подсистемой Linux	11
1.3.1 Структура struct sk_buff	12
1.3.2 Структура struct request_sock	14
1.3.3 Структура struct sock	16
1.3.4 Структура struct inet_sock	16
1.3.5 Структура struct inet_request_sock	17
1.3.6 Структура struct inet_connection_sock	17
1.3.7 Структура struct tcp_timewait_sock	18
1.4 Сетевое устройство в Linux	18
1.5 Фреймворк Linux Express Data Path (XDP)	20
1.5.1 Структура struct xdp_md	20
1.5.2 Структура struct ethhdr	21
1.5.3 Структура struct iphdr	21
1.5.4 Структура struct tcphdr	21
1.6 DDoS атака SYN Flood	22
1.7 Метод защиты от SYN Flood с помощью TCP Cookies	22
2 Конструкторский раздел	24
3 Технологический раздел	28
3.1 Выбор языка и среды программирования	28
3.2 Описание разработанного XDP модуля	28
3.3 Сборка и загрузка XDP модуля	29
4 Исследовательский раздел	30

ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33
ПРИЛОЖЕНИЕ А Исходный код модуля	34
ПРИЛОЖЕНИЕ Б Файл сборки Makefile	40
ПРИЛОЖЕНИЕ В Исходный код скрипта для тестирования	41

ВВЕДЕНИЕ

Сетевая атака типа «Отказ в обслуживании» (DDoS) на компьютерную систему приводит к тому, что легитимные клиенты не могут получить доступ к ресурсу.

DDoS атака заключается в послылке атакуемому серверу большого числа запросов с множества компьютеров, называемого ботнет-сетью.

Одна из разновидностей DDoS атак – атака SYN Flood. SYN Flood – это массированная атака, которая направлена на исчерпание сетевых ресурсов сервера.

Согласно отчету компании Cloudflare за 2025 год, за последние два года число массированных DDoS атак возросло практически вдвое [1].

Цель курсовой работы – модуля ядра Linux для защиты от DDoS атаки типа SYN Flood с помощью TCP Cookies.

1 Аналитический раздел

1.1 Постановка задачи

В соответствии с заданием на курсовую работу требуется разработать модуль ядра Linux для защиты от DDoS атаки типа SYN Flood с помощью TCP Cookies.

Для выполнения работы требуется решить следующие задачи:

1. изучить протокол TCP и процесс установки соединения путем трехстороннего рукопожатия;
2. изучить процесс обработки сетевых кадров системой, описать используемые структуры ядра;
3. описать технологию Express Data Path (XDP) для обработки сетевого кадра на ранней стадии;
4. описать принцип атаки SYN Flood и методы борьбы с ней на основе TCP Cookies;
5. разработать алгоритмы, необходимые для реализации модуля;
6. разработать модуль;
7. провести исследование разработанного модуля.

1.2 Протокол TCP

Transmission Control Protocol (TCP) – это протокол транспортного уровня сетевой модели OSI [2]. Протокол TCP гарантирует надежную доставку сегментов данных с сохранением их порядка и повторную передачу недоставленных сегментов.

1.2.1 Заголовок TCP сегмента

Заголовок TCP сегмента включает поля:

- Source Port и Destination Port — номера портов отправителя и получателя;

- Sequence Number (seq) — начальный порядковый номер для данных, которые отправит клиент;
- Acknowledgment Number (ack) — номер следующего ожидаемого байта клиента;
- Data Offset — длина заголовка в словах;
- Flags — флаги (SYN, ACK, FIN, RST, PSH, URG);
- Window Size — размер окна, используемый для управления потоком;
- Checksum — контрольная сумма TCP сегмента;
- Urgent Pointer — указатель срочных данных;
- Options — дополнительные параметры.

На рисунке 1.1 приведен формат TCP заголовка.



Рисунок 1.1 – Формат TCP заголовка

При установке TCP соединения важнейшую роль имеют значения полей seq, ack и флаги SYN и ACK.

1.2.2 Трехстороннее рукопожатие

Для обеспечения надежной доставки данных перед началом передачи сегментов данных необходимо установить соединение путем трехстороннего рукопожатия [2].

SYN запрос

Клиент делает системный вызов `connect()`, который переводит сокет в состояние `SYN_SENT` [3] и посылает серверу SYN запрос. Заголовок отправленного сегмента обязательно содержит установленный флаг SYN и начальный порядковый номер данных в поле `seq`.

SYN-ACK ответ

Сокет сервера находится в состоянии `TCP_LISTEN` в результате системного вызова `listen`. При получении SYN запроса сокет сервера меняет состояние на `SYN_RECV`, ядро выделяет ресурсы для хранения данных о полуоткрытом соединении, в частности структуру `struct sock`, которая помещается в очередь полуоткрытых соединений SYN Queue. Сервер отправляет клиенту с SYN-ACK ответ, заголовок которого содержит:

- установленные флаги SYN и ACK;
- поле `seq` и поле `ack` инициализированное значением на единицу больше, чем значение `seq` из SYN запроса.

ACK запрос

Для завершения установки соединения клиент, получив SYN-ACK ответ, отправляет серверу ACK запрос. Заголовок ACK запроса содержит:

- установленный флаг ACK;
- поле `seq`, инициализированное значением поля `ack` из SYN-ACK ответа;
- поле `ack`, инициализированное значением поля `seq` из SYN-ACK ответа, увеличенным на единицу.

В результате установки соединения сокет клиент и сервер устанавливают статус `ESTABLISHED`, данные о соединении помещаются в очередь принятых соединений `Accept Queue`.

На рисунке 1.2 приведена схема трехстороннего рукопожатия.

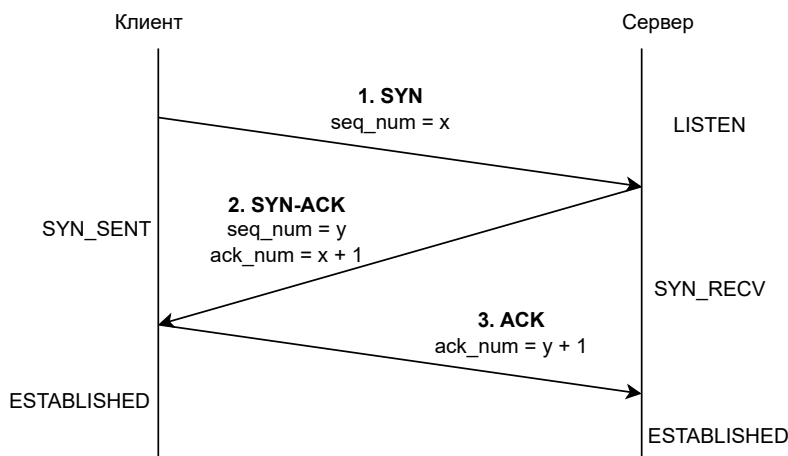


Рисунок 1.2 – Схема трехстороннего рукопожатия

1.3 Обработка SYN запроса сетевой подсистемой Linux

Кадр, поступивший на сетевую карту, поступает в обработку драйверу NIC, далее с помощью механизма прямого доступа к памяти (DMA) данные помещаются в кольцевой буфер `RX-ring`. Кольцевой буфер `RX-ring` хранит дескрипторы кадров данных. Дескриптор кадра данных содержит адрес в памяти ядра, куда помещены данные из кадра, а также дополнительную информацию. Листинг 1.1 содержит объявление структуры дескриптора для драйвера сетевой карты Intel e1000.

Листинг 1.1 – Дескриптор кадра данных для драйвера Intel e1000

```

1 struct e1000_rx_desc {
2     __le64 buffer_addr;
3     __le16 length;
4     __le16 csum;
5     u8 status;
6     u8 errors;
7     __le16 special;
8 };
  
```


Для уведомления системы о поступлении кадра, инициируется аппаратное прерывание. Начиная с версии 2.4 ядра добавлена технология опроса NAPI, которая предназначена для уменьшения числа прерываний от сетевых устройств [4].

Для дальнейшей обработки кадра сетевой подсистемой ядро выделяет страницы адресного пространства ядра и инициализирует структуры для хранения данных кадра.

1.3.1 Структура `struct sk_buff`

Структура `struct sk_buff` – это структура метаданных сетевого кадра, которая содержит указатели на буфер данных и буфер разделяемой информации.

Драйвер вызывает функцию ядра `dev_alloc_skb()`, которая вызывает функцию `netdev_alloc_skb()`, а она в свою очередь функцию `__netdev_alloc_skb()`. Функция `__netdev_alloc_skb()` создает экземпляр структуры `struct sk_buff` и инициализирует его поля данными пакета.

Листинг 1.2 содержит исходный код упомянутых выше функций. Листинг 1.2 – Исходный код функций `netdev_alloc_skb()`, `netdev_alloc_skb()`, `__netdev_alloc_skb()`

```
1 static inline struct sk_buff *dev_alloc_skb(unsigned int length)
2 {
3     return netdev_alloc_skb(NULL, length);
4 }
5
6 static inline struct sk_buff *netdev_alloc_skb(struct
7     net_device *dev,
8     unsigned int length)
9 {
10    return __netdev_alloc_skb(dev, length, GFP_ATOMIC);
11 }
12 struct sk_buff *__netdev_alloc_skb(struct net_device *dev,
13     unsigned int len,
14     gfp_t gfp_mask)
15 {
16    struct page_frag_cache *nc;
17    struct sk_buff *skb;
18    bool pfmemalloc;
```

```

18  void *data;
19  len += NET_SKB_PAD;
20  if (len <= SKB_WITH_OVERHEAD(SKB_SMALL_HEAD_CACHE_SIZE) ||
21      len > SKB_WITH_OVERHEAD(PAGE_SIZE) ||
22      (gfp_mask & (__GFP_DIRECT_RECLAIM | GFP_DMA))) {
23      skb = __alloc_skb(len, gfp_mask, SKB_ALLOC_RX,
24                        NUMA_NO_NODE);
25      if (!skb)
26          goto skb_fail;
27      goto skb_success;
28  }
29  // ...
30  skb = __build_skb(data, len);
31  if (unlikely(!skb)) {
32      skb_free_frag(data);
33      return NULL;
34  }
35  if (pfmemalloc)
36      skb->pfmemalloc = 1;
37      skb->head_frag = 1;
38  skb_success:
39      skb_reserve(skb, NET_SKB_PAD);
40      skb->dev = dev;
41
42  skb_fail:
43      return skb;
44  }
45  EXPORT_SYMBOL(__netdev_alloc_skb);

```

Листинг 1.3 содержит объявление структуры `struct sk_buff`.

Листинг 1.3 – Объявление структуры `struct sk_buff`

```

1  struct sk_buff {
2      union {
3          struct {
4              struct sk_buff    *next;
5              struct sk_buff    *prev;
6
7              union {
8                  struct net_device *dev;
9

```

```

10     unsigned long    dev_scratch;
11 };
12 };
13 struct rb_node      rbnode; /* used in netem, ip4 defrag, and
    tcp stack */
14 struct list_head    list;
15 struct llist_node  ll_node;
16 };
17 struct sock        *sk;
18 union {
19     ktime_t        tstamp;
20     u64            skb_mstamp_ns; /* earliest departure time */
21 };
22 };

```

1.3.2 Структура struct request_sock

Структура `struct request_sock` служит для хранения информации о полуоткрытом соединении.

Функция ядра `inet_reqsk_alloc()` создает экземпляр структуры `struct request_sock`, в которую также копируется некоторая информация из `struct sk_buff`. Листинг 1.4 содержит объявление структуры `struct request_sock`.

Листинг 1.4 – Объявление структуры `struct request_sock`

```

1 struct request_sock {
2     struct sock_common    __req_common;
3 #define rsk_refcnt        __req_common.skc_refcnt
4 #define rsk_hash          __req_common.skc_hash
5 #define rsk_listener      __req_common.skc_listener
6 #define rsk_window_clamp  __req_common.skc_window_clamp
7 #define rsk_rcv_wnd       __req_common.skc_rcv_wnd
8
9     struct request_sock    *dl_next;
10    u16                    mss;
11    u8                    num_retrans; /* number of retransmits */
12    u8                    syncookie:1; /* True if
13                                     * 1) tcpopts needs to be encoded in
14                                     *    TS of SYN+ACK
15                                     * 2) ACK is validated by BPF kfunc.
16                                     */

```

```

17  u8          num_timeout:7; /* number of timeouts */
18  u32         ts_recent;
19  struct timer_list    rsk_timer;
20  const struct request_sock_ops *rsk_ops;
21  struct sock          *sk;
22  struct saved_syn      *saved_syn;
23  u32                 secid;
24  u32                 peer_secid;
25  u32                 timeout;
26  };

```

Листинг 1.5 содержит исходный код функции `inet_reqsk_alloc()`.

Листинг 1.5 – Исходный код функции `inet_reqsk_alloc()`.

```

1  struct request_sock *inet_reqsk_alloc(const struct
    request_sock_ops *ops,
2      struct sock *sk_listener,
3      bool attach_listener)
4  {
5      struct request_sock *req = reqsk_alloc(ops, sk_listener,
6          attach_listener);
7
8      if (req) {
9          struct inet_request_sock *ireq = inet_rsk(req);
10         ireq->ireq_opt = NULL;
11         atomic64_set(&ireq->ir_cookie, 0);
12         ireq->ireq_state = TCP_NEW_SYN_RECV;
13         write_pnet(&ireq->ireq_net, sock_net(sk_listener));
14         ireq->ireq_family = sk_listener->sk_family;
15         req->timeout = TCP_TIMEOUT_INIT;
16     }
17
18     return req;
19 }

```

Экземпляры структуры `struct request_sock` содержит данные о полукоткрытых соединениях, именно они помещаются в очередь полукоткрытых соединений SYN Queue. Это происходит в функции `inet_csk_reqsk_queue_hash_add`, исходный код которой приведен на листинге 1.6

Листинг 1.6 – Исходный код функции `inet_csk_reqsk_queue_hash_add()`.

```

1 | bool inet_csk_reqsk_queue_hash_add(struct sock *sk, struct
   |     request_sock *req,
2 |         unsigned long timeout)
3 | {
4 |     if (!reqsk_queue_hash_req(req, timeout))
5 |         return false;
6 |
7 |     inet_csk_reqsk_queue_added(sk);
8 |     return true;
9 | }

```

1.3.3 Структура **struct sock**

Структура **struct sock** хранит информацию о сокете. Листинг 1.7 содержит объявление структуры **struct sock**.

Листинг 1.7 – Объявление структуры **struct sock**

```

1 | struct sock {
2 |     struct sock_common      __sk_common;
3 |     struct sk_buff_head     sk_receive_queue;
4 |     struct sk_buff_head     sk_write_queue;
5 |     socket_lock_t          sk_lock;
6 |     int                     sk_rcvbuf;
7 |     int                     sk_sndbuf;
8 |     struct proto             *sk_prot;
9 |     struct inet_sock        *sk_inet;
10 |
11 | };

```

Данную структуру создает функция ядра **__sock_create()**.

1.3.4 Структура **struct inet_sock**

Структура **struct inet_sock** содержит данные, необходимые для работы протокола Internet Protocol (IP). Листинг 1.8 содержит объявление структуры **struct inet_sock**.

Листинг 1.8 – Объявление структуры **struct inet_sock**

```

1 | struct inet_sock {
2 |     struct sock             sk;
3 |     __be32                  inet_daddr;
4 |     __be32                  inet_rcv_saddr;

```

```

5     __be16                inet_dport;
6     __be16                inet_num;
7     __u32                  inet_saddr;
8     // ..
9 };

```

1.3.5 Структура struct inet_request_sock

Структура `struct inet_request_sock` содержит данные, необходимые для работы протокола Internet Protocol (IP) на этапе трехстороннего рукопожатия. Листинг 1.9 содержит объявление структуры `struct inet_request_sock`.

Листинг 1.9 – Объявление структуры `struct inet_request_sock`

```

1 struct inet_request_sock {
2     struct request_sock    req;
3     __be16                loc_port;
4     __be32                loc_addr;
5     __be16                rmt_port;
6     __be32                rmt_addr;
7
8 };

```

1.3.6 Структура struct inet_connection_sock

Структура `struct inet_connection_sock` содержит очереди принятия запросов, таймеры повторной передачи и другие поля, необходимые для корректной работы протоколов транспортного уровня.

Листинг 1.10 содержит объявление структуры `struct inet_connection_sock`.

Листинг 1.10 – Объявление структуры `struct inet_connection_sock`

```

1 struct inet_connection_sock {
2     struct inet_sock        icsk_inet;
3     struct request_sock_queue icsk_accept_queue;
4     inet_csk_auth_fn_t      *icsk_af_ops;
5     struct timer_list        icsk_retransmit_timer;
6     struct timer_list        icsk_delack_timer;
7     __u8                    icsk_pending;
8
9 };

```

1.3.7 Структура `struct tcp_timewait_sock`

Структура `struct tcp_timewait_sock` представляет короткоживущий сокет, используемый для корректного завершения TCP соединения. Она имеет состояние `TCP_TIME_WAIT` после закрытия соединения и служит для обработки возможных повторно пришедших сегментов.

Листинг 1.11 содержит объявление структуры `struct inet_timewait_sock`.

Листинг 1.11 – Объявление структуры `struct tcp_timewait_sock`

```
1 struct tcp_timewait_sock {  
2     struct inet_timewait_sock tw_sk;  
3     u32      tw_rcv_nxt;  
4     u32      tw_snd_nxt;  
5     u32      tw_ts_recent;  
6  
7 };
```

Такими образом, для обработки одного SYN запроса необходимо создать и проинициализировать экземпляры семи структур – это значительные затраты страниц адресного пространства ядра.

1.4 Сетевое устройство в Linux

Сетевое устройство Linux – это структура ядра `struct net_device`, которая представляет физический или виртуальный сетевой интерфейс.

Идентификатором сетевого интерфейса является его имя (`eth0`, `lo` и другие). В ядре сетевой интерфейс представлен структурой `struct net_device`, которая приведена на листинге 1.12.

Листинг 1.12 – Структура ядра `struct net_device`.

```
1 struct net_device {  
2     struct_group(priv_flags_fast ,  
3         unsigned long    priv_flags:32;  
4         unsigned long    lltx:1;  
5         unsigned long    netmem_tx:1;  
6     );  
7     const struct net_device_ops *netdev_ops;  
8     const struct header_ops *header_ops;
```

```

9  struct netdev_queue *_tx;
10 struct netdev_tc_txq  tc_to_txq[TC_MAX_QUEUE];
11 struct xps_dev_maps __rcu *xps_maps[XPS_MAPS_MAX];
12 union {
13     struct pcpu_lstats __percpu *lstats;
14     struct pcpu_sw_netstats __percpu *tstats;
15     struct pcpu_dstats __percpu *dstats;
16 };
17 unsigned long    state;
18 unsigned int     flags;
19 unsigned short   hard_header_len;
20 netdev_features_t features;
21 struct inet6_dev __rcu *ip6_ptr;
22 struct list_head  napi_list;
23 struct list_head  unreg_list;
24 struct list_head  close_list;
25 struct list_head  ptype_all;
26 xdp_features_t    xdp_features;
27 const struct xdp_metadata_ops *xdp_metadata_ops;
28 const struct xsk_tx_metadata_ops *xsk_tx_metadata_ops;
29 unsigned short   gflags;
30 };

```

Сетевой интерфейс может соответствовать реальному устройству (NIC) или виртуальному (veth, loopback).

1.5 Фреймворк Linux Express Data Path (XDP)

XDP – это фреймворк ядра Linux, который позволяет связать загружаемый модуль с определенным сетевым интерфейсом и реализовать перехват кадров до выделения системой ресурсов для создания и инициализации структур. Данный фреймворк доступен в версии ядра Linux 4.8 выше, также он должен поддерживаться драйвером сетевой карты.

Перехваченный кадр может быть пропущен (XDP_PASS), отброшен (XDP_DROP), отправлен обратно получателю (XDP_TX) или перенаправлен на другой сетевой интерфейс (XDP_REDIRECT).

Модуль, загружаемый с помощью XDP, обязательно должен иметь точку входа с сигнатурой, представленной на листинге 1.13.

Листинг 1.13 – Точка входа в модуль, загружаемый с помощью XDP

```
1
2 SEC("NAME")
3 int xdp_main(struct xdp_md* ctx) {
4     return 0;
5 }
```

1.5.1 Структура struct xdp_md

Структура struct xdp_md инициализируется драйвером сетевой карты и содержит информацию о полученном кадре, ее объявление приведено на листинге 1.14.

Листинг 1.14 – Структура struct xdp_md

```
1 struct xdp_md {
2     __u32 data;
3     __u32 data_end;
4     __u32 data_meta;
5     /* Below access go through struct xdp_rxq_info */
6     __u32 ingress_ifindex; /* rxq->dev->ifindex */
7     __u32 rx_queue_index; /* rxq->queue_index */
8
9     __u32 egress_ifindex; /* txq->dev->ifindex */
10 };
```

Поля data и data_end – это адреса физической памяти ядра, между которыми заключены данные кадра.

1.5.2 Структура struct ethhdr

Структура struct ethhdr содержит данные для адресации кадра на канальном уровне, ее объявление приведено на листинге 1.15.

Листинг 1.15 – Структура struct ethhdr

```
1 struct ethhdr {  
2     unsigned char h_dest[ETH_ALEN]; /* destination eth addr */  
3     unsigned char h_source[ETH_ALEN]; /* source ether addr */  
4     __be16      h_proto; /* packet type ID field */  
5 } __attribute__((packed));
```

1.5.3 Структура struct iphdr

Структура struct iphdr содержит данные для адресации кадра на канальном уровне, ее объявление приведено на листинге 1.16.

Листинг 1.16 – Структура struct iphdr

```
1 struct iphdr {  
2     __u8  tos;  
3     __be16 tot_len;  
4     __be16 id;  
5     __be16 frag_off;  
6     __u8  ttl;  
7     __u8  protocol;  
8     __sum16 check;  
9     __be32 saddr;  
10    __be32 daddr;  
11 };
```

1.5.4 Структура struct tcphdr

Структура struct tcphdr содержит данные для адресации кадра на канальном уровне, ее объявление приведено на листинге 1.17.

Листинг 1.17 – Структура struct tcphdr

```
1 struct tcphdr {  
2     __u16 source;  
3     __u16 dest;  
4     __u32 seq;  
5     __u32 ack_seq;  
6     union {
```

```
7 |         __u16 flags ;
8 |     };
```

1.6 DDoS атака SYN Flood

Distributed Denial Of Service (DDoS) – это атака типа «Отказ в обслуживании» [5]. Цель атаки – добиться отказа сервиса или значительного повышения времени обслуживания клиентов.

SYN Flood – это DDoS атака, которая заключается в посылке серверу множества SYN запросов с огромного количества клиентов, называемых ботнет-сетью [6]. Цель атаки SYN Flood – заполнить очередь SYN Queue сервера.

В ядре SYN Queue представлена структурой `struct inet_ehash_bucket`, объявление которой приведено на листинге 1.18.

Листинг 1.18 – Структура `struct inet_ehash_bucket`

```
1 | struct inet_ehash_bucket {
2 |     struct hlist_nulls_head chain; };
```

Очередь полуоткрытых соединений SYN Queue сервера имеет фиксированный размер, который по умолчанию равен 4096 и может быть просмотрен с помощью утилиты `sysctl`, приведенной на листинге 1.19.

Листинг 1.19 – Утилита `sysctl` для просмотра размера очереди SYN Queue.

```
1 | nikita@nikita-VirtualBox:$ sysctl net.core.somaxconn
2 | net.core.somaxconn = 4096
```

Пока очередь заполнена, все поступающие серверу SYN запросы отбрасываются. В результате такой атаки легитимные клиенты не могут быть обслужены.

Особенность данного типа атаки заключается в том, что ботнет клиенты не дожидаются получения ACK ответа сервера.

1.7 Метод защиты от SYN Flood с помощью TCP Cookies

TCP Cookies — это метод защиты от атаки типа SYN Flood.

Метод заключается в отказе от выделения системой ресурсов для хранения данных о полуоткрытых соединениях.

Сервер не выделяет ресурсы, а сразу посылает клиенту SYN-ACK ответ, в поле Sequence Number которого помещается cookie – хеш, вычисленный на основе секретного ключа сервера и адреса клиента.

При получении ACK запроса сервер выполняет декодирование поля Sequence Number и его проверку cookie. Если данные в поле верны, то есть были подписаны сервером ранее, то клиент идентифицируется как легитимный и система выделяет ресурсы для соединения.

Преимущество метода – противодействие SYN Flood атаке, позволяющее легитимным клиентам получить доступ к ресурсу.

Недостаток метода заключается в том, что с применением TCP Cookies становится невозможным использование данных из поля Options заголовка TCP сегмента. Это происходит из-за того, что данные параметры передаются в SYN запросе, ресурсы для которого не выделяются.

Вывод

В аналитическом разделе приведены теоретические сведения о протоколе TCP, перечислены основные функции и структуры ядра, задействованные в обработке поступившего на сетевой интерфейс кадра данных. Описан принцип работы DDoS атаки SYN Flood и метода противодействия ей – TCP Cookies. Рассмотрена технология ядра Linux XDP, которая позволяет произвести проверку входящего кадра данных до выделения системой ресурсов.

2 Конструкторский раздел

На рисунке 2.1 представлена функциональная модель системы в нотации IDEF0 нулевого уровня.

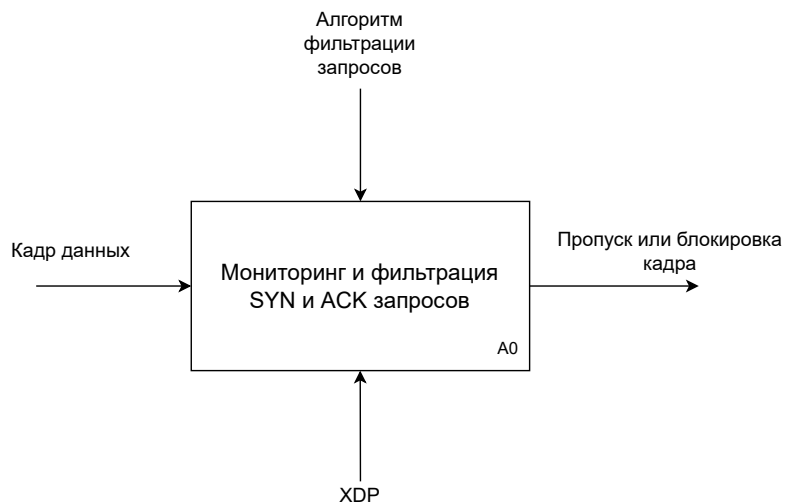


Рисунок 2.1 – Функциональная модель системы в нотации IDEF0

На рисунке 2.2 приведена схема алгоритма работы модуля, реализующего защиту сетевого интерфейса от SYN Flood атаки.

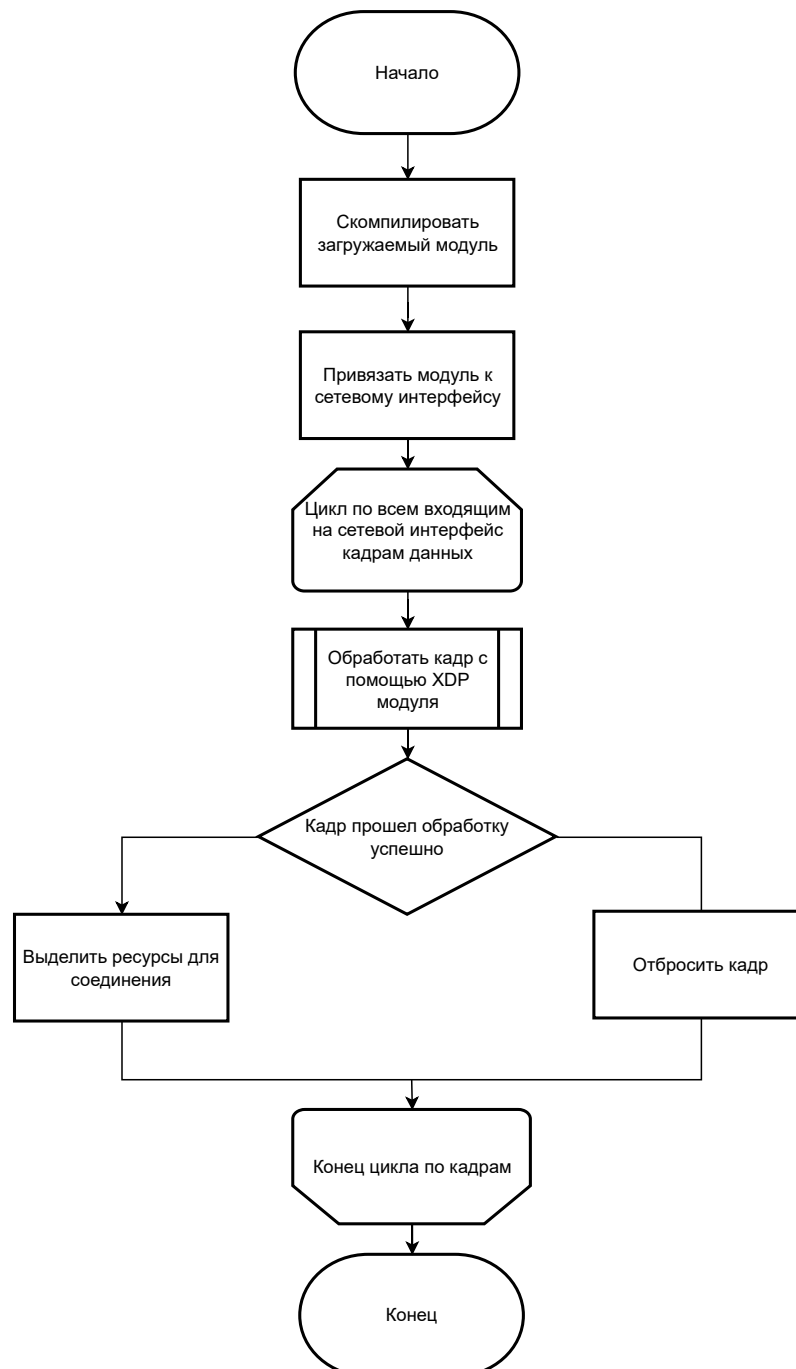


Рисунок 2.2 – Схема алгоритма работы модуля, реализующего защиту сетевого интерфейса от SYN Flood атаки

На рисунке 2.3 приведена схема алгоритма обработки поступившего на сетевой интерфейс кадра данных.

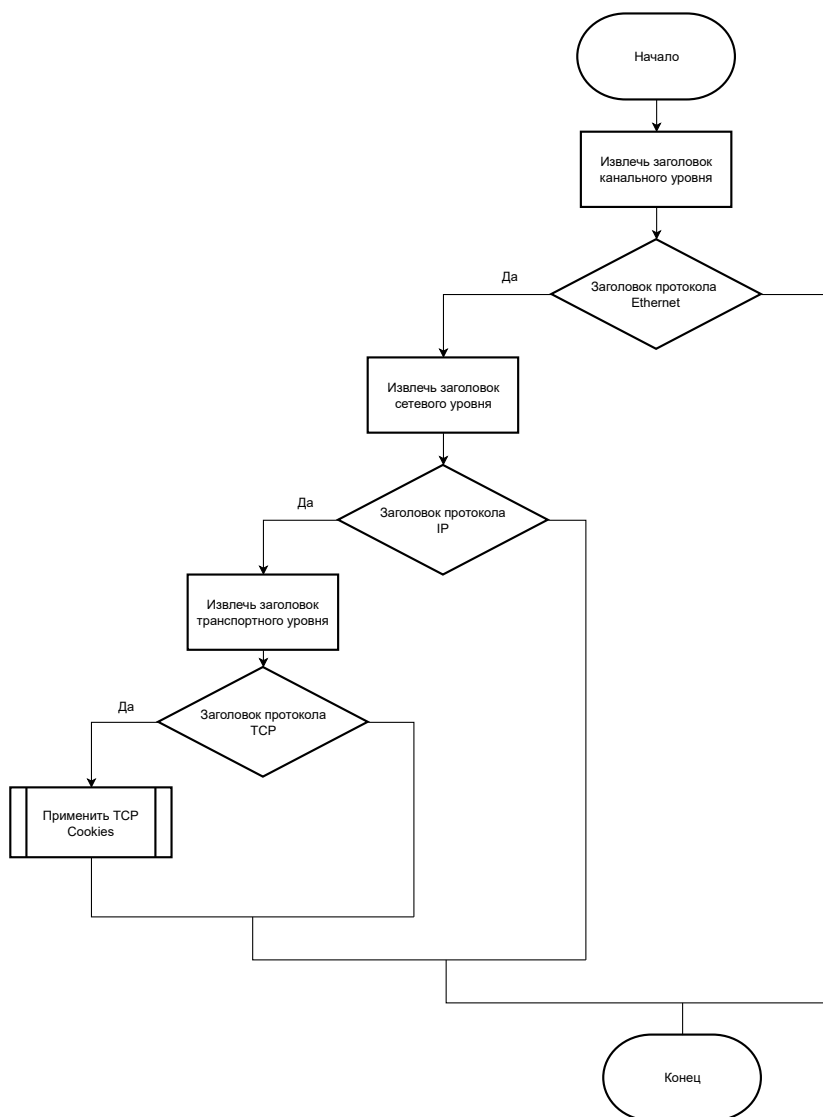


Рисунок 2.3 – Схема алгоритма обработки поступившего на сетевой интерфейс кадра данных

На рисунке 2.4 приведена схема алгоритма обработки TCP сегмента с использованием TCP Cookie

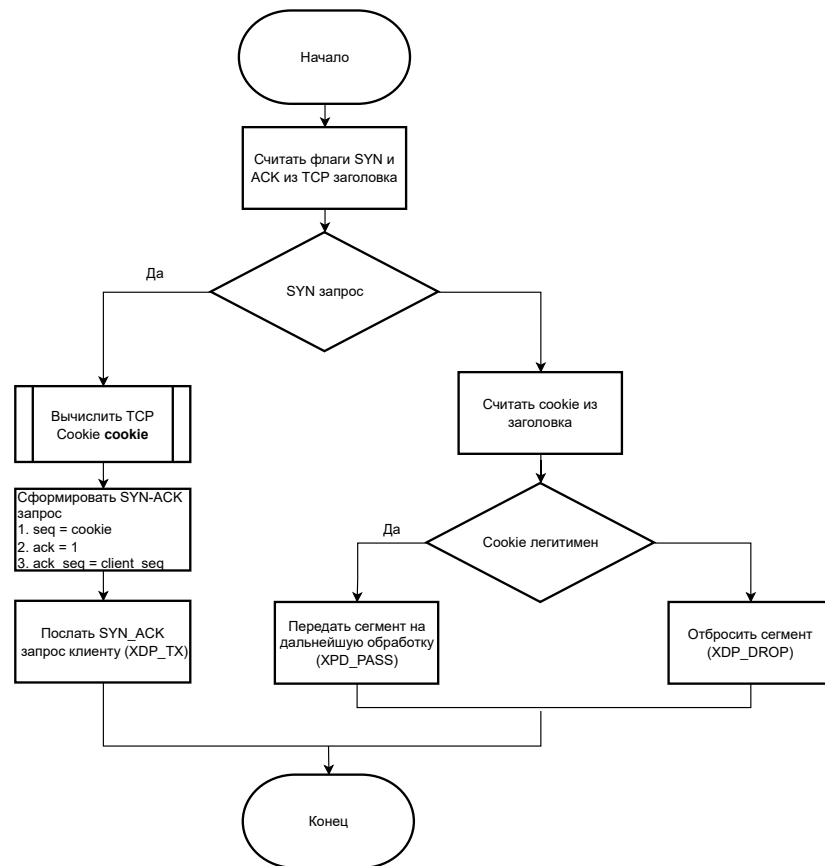


Рисунок 2.4 – Схема алгоритма обработки TCP сегмента с использованием TCP Cookie

3 Технологический раздел

3.1 Выбор языка и среды программирования

Для разработки был использован язык программирования Си, так как на этом языке реализованы все драйверы и модули ОС Linux.

Для компиляции XDP модуля использовался компилятор clang.

Реализованный модуль поддерживается версиями ядра, начиная с 4.8.

В качестве среды разработки была выбрана программа VSCode.

3.2 Описание разработанного XDP модуля

Разработанный модуль ядра реализует защиту определенного сетевого интерфейса от DDoS атаки SYN Flood с помощью TCP Cookies.

Модуль перехватывает кадры данных после их обработки драйвером NIC. Кадр деинкапсулируется для определения типа протокола транспортного уровня. Если кадр содержит в себе TCP сегмент и является SYN запросом, то модуль формирует TCP Cookie на основе его IP адреса и секретного ключа и отправляет SYN-ACK ответ клиенту без выделения ресурсов. Если кадр является ACK запросом, то модуль выполняет проверку TCP Cookie. Если TCP Cookie корректен, то клиент считается легитимным и соединение устанавливается, в противном случае пакет отбрасывается.

Точкой входа в XDP модуль является функция `int xdp_main()`, исходный код которой приведен на листинге В.1.

Листинг 3.1 – Точка входа в XDP модуль

```
1 int xdp_main(struct xdp_md* ctx) {  
2     struct Packet packet;  
3     packet.ctx = ctx;  
4     struct ethhdr* ether = (struct ethhdr*)(void*)ctx->data;  
5     if (((void*)(ether + 1) > (void*)ctx->data_end) {  
6         return XDP_PASS;    }  
7     packet.ether = ether;  
8     return process_ether(&packet);  
9 }
```

3.3 Сборка и загрузка XDP модуля

Для сборки модуля реализован Makefile, исходный код которого расположен в приложении. Результат сборки – объектный файл модуля.

Для загрузки XDP модуля требуется использовать утилиту `ip`. При загрузке модуля необходимо указать, к какому сетевому устройству (по имени) будет привязан модуль. Листинг 3.2 содержит команду для загрузки XDP модуля и его привязки к сетевому устройству с именем `eth0`.

Листинг 3.2 – Загрузка XDP модуля

```
1 ip link set dev eth0 xdp obj xdp.o
```

Полный исходный код реализованного XDP модуля находится в приложении.

4 Исследовательский раздел

Для исследования работоспособности разработанного модуля реализован скрипт командной оболочки, который содержит команды для загрузки модуля, создания тестового стенда и просмотра отладочной печати из системного журнала. Исходный код скрипта приведен в приложении.

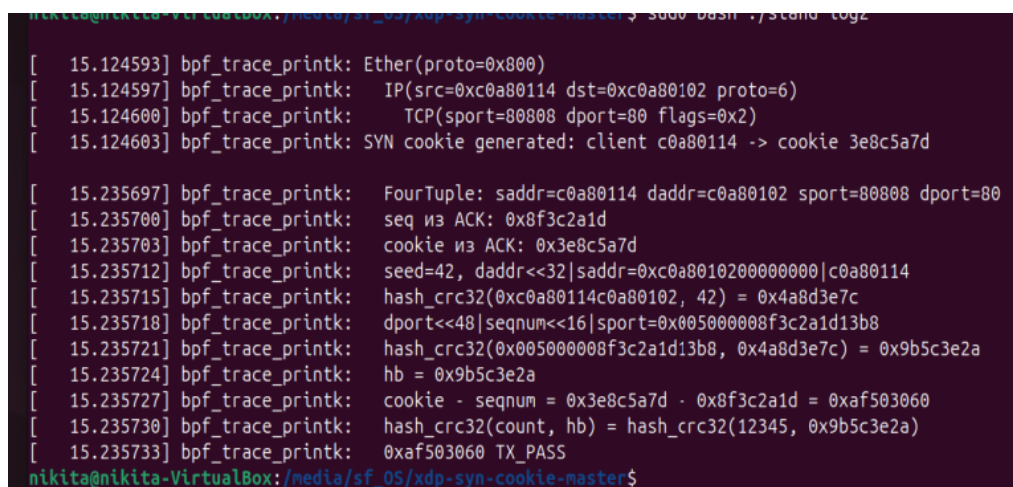
Команда `sudo ./stand up` создает тестовый стенд.

Данная команда создает два пространства сетевых имен (net namespace) и виртуальное соединение между ними (Virtual Ethernet). Данные действия нужны для того, чтобы симитировать отправку на сервер запросов с удаленных хостов, используя при этом один физический хост.

Команда `sudo ./stand attach` загружает XDP модуль в ядро и связывает его с сетевым устройством.

Отладочная печать модуля выводится в системный журнал, расположенный в `/sys/kernel/debug/tracing/options/trace_printk`.

На рисунке 4.1 приведен результат обработки SYN запроса от легитимного клиента.



```
nikita@nikita-VirtualBox:~/media/sf_05/xdp-syn-cookie-master$ sudo bash ./stand logz
[ 15.124593] bpf_trace_printk: Ether(proto=0x800)
[ 15.124597] bpf_trace_printk: IP(src=0xc0a80114 dst=0xc0a80102 proto=6)
[ 15.124600] bpf_trace_printk: TCP(sport=80808 dport=80 flags=0x2)
[ 15.124603] bpf_trace_printk: SYN cookie generated: client c0a80114 -> cookie 3e8c5a7d

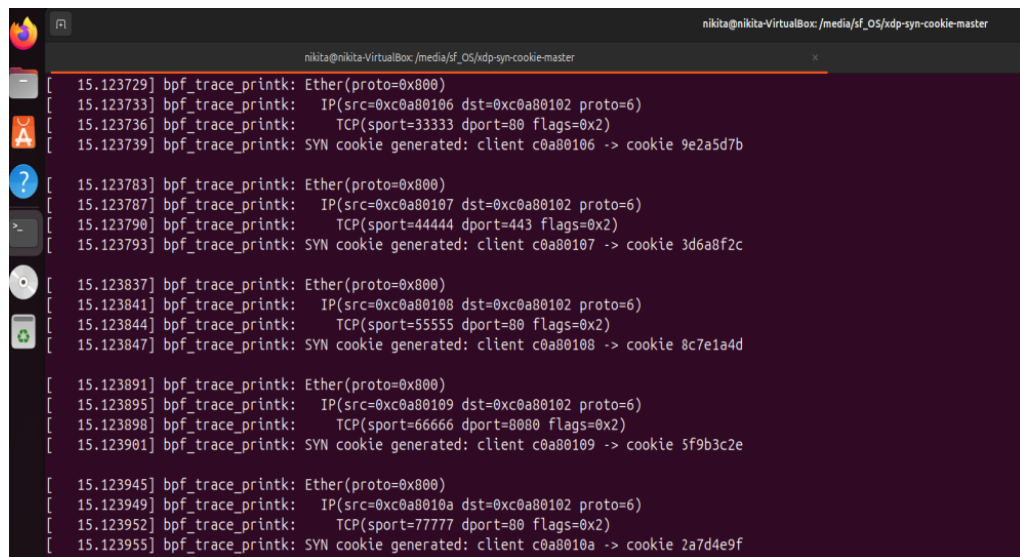
[ 15.235697] bpf_trace_printk: FourTuple: saddr=c0a80114 daddr=c0a80102 sport=80808 dport=80
[ 15.235700] bpf_trace_printk: seq из ACK: 0x8f3c2a1d
[ 15.235703] bpf_trace_printk: cookie из ACK: 0x3e8c5a7d
[ 15.235712] bpf_trace_printk: seed=42, daddr<<32|saddr=0xc0a8010200000000|c0a80114
[ 15.235715] bpf_trace_printk: hash_crc32(0xc0a80114c0a80102, 42) = 0x4a8d3e7c
[ 15.235718] bpf_trace_printk: dport<<48|seqnum<<16|sport=0x005000008f3c2a1d13b8
[ 15.235721] bpf_trace_printk: hash_crc32(0x005000008f3c2a1d13b8, 0x4a8d3e7c) = 0x9b5c3e2a
[ 15.235724] bpf_trace_printk: hb = 0x9b5c3e2a
[ 15.235727] bpf_trace_printk: cookie - seqnum = 0x3e8c5a7d - 0x8f3c2a1d = 0xaf503060
[ 15.235730] bpf_trace_printk: hash_crc32(count, hb) = hash_crc32(12345, 0x9b5c3e2a)
[ 15.235733] bpf_trace_printk: 0xaf503060 TX_PASS
nikita@nikita-VirtualBox:~/media/sf_05/xdp-syn-cookie-master$
```

Рисунок 4.1 – Легитимный клиент

С помощью утилиты **hping3** произведен SYN Flood атаку. На рисунках 4.2 и 4.3 приведены результаты исследования

```
nikita@nikita-VirtualBox:/media/sf_OS/xdp-syn-cookie-master$ sudo hping3 --flood -A -s 1111 -p 2222 192.0.2.1
[sudo] password for nikita:
HPING 192.0.2.1 (xdp-local 192.0.2.1): A set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 192.0.2.1 hping statistic ---
11184560 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
nikita@nikita-VirtualBox:/media/sf_OS/xdp-syn-cookie-master$ SS
```

Рисунок 4.2 – SYN Flood (Ботнет)



```
nikita@nikita-VirtualBox:/media/sf_OS/xdp-syn-cookie-master
[ 15.123729] bpf_trace_printk: Ether(proto=0x800)
[ 15.123733] bpf_trace_printk: IP(src=0xc0a80106 dst=0xc0a80102 proto=6)
[ 15.123736] bpf_trace_printk: TCP(sport=33333 dport=80 flags=0x2)
[ 15.123739] bpf_trace_printk: SYN cookie generated: client c0a80106 -> cookie 9e2a5d7b

[ 15.123783] bpf_trace_printk: Ether(proto=0x800)
[ 15.123787] bpf_trace_printk: IP(src=0xc0a80107 dst=0xc0a80102 proto=6)
[ 15.123790] bpf_trace_printk: TCP(sport=44444 dport=443 flags=0x2)
[ 15.123793] bpf_trace_printk: SYN cookie generated: client c0a80107 -> cookie 3d6a8f2c

[ 15.123837] bpf_trace_printk: Ether(proto=0x800)
[ 15.123841] bpf_trace_printk: IP(src=0xc0a80108 dst=0xc0a80102 proto=6)
[ 15.123844] bpf_trace_printk: TCP(sport=55555 dport=80 flags=0x2)
[ 15.123847] bpf_trace_printk: SYN cookie generated: client c0a80108 -> cookie 8c7e1a4d

[ 15.123891] bpf_trace_printk: Ether(proto=0x800)
[ 15.123895] bpf_trace_printk: IP(src=0xc0a80109 dst=0xc0a80102 proto=6)
[ 15.123898] bpf_trace_printk: TCP(sport=66666 dport=8080 flags=0x2)
[ 15.123901] bpf_trace_printk: SYN cookie generated: client c0a80109 -> cookie 5f9b3c2e

[ 15.123945] bpf_trace_printk: Ether(proto=0x800)
[ 15.123949] bpf_trace_printk: IP(src=0xc0a8010a dst=0xc0a80102 proto=6)
[ 15.123952] bpf_trace_printk: TCP(sport=77777 dport=80 flags=0x2)
[ 15.123955] bpf_trace_printk: SYN cookie generated: client c0a8010a -> cookie 2a7d4e9f
```

Рисунок 4.3 – SYN Flood (Обработка сервером)

Вывод

Исследование разработанного модуля показало, что он полностью отвечает техническому заданию и корректно решает все поставленные задачи

ЗАКЛЮЧЕНИЕ

В ходе работы решены следующие задачи:

- 1) описан протокол TCP;
- 2) описан процесс обработки кадров сетевой подсистемой Linux, приведены соответствующие структуры ядра;
- 3) спроектированы схемы алгоритмов работы модуля для защиты сетевого устройства от DDoS атаки SYN Flood;
- 4) разработан XDP модуль;
- 5) модуль протестирован.

Цель курсовой работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Отчет по DDoS атакам Cloudflare [Электронный ресурс]. — Режим доступа: <https://blog.cloudflare.com/ddos-threat-report-for-2025-q2/> (дата обращения: 29.10.2025).
2. Протокол TCP [Электронный ресурс]. — Режим доступа: https://en.wikipedia.org/wiki/Transmission_Control_Protocol (дата обращения: 29.10.2025).
3. Исходный код ядра Linux [Электронный ресурс]. — Режим доступа: <https://elixir.bootlin.com/linux/v6.18.2/source> (дата обращения: 29.10.2025).
4. *Рязанова Н. Ю.* АНАЛИЗ СРЕДСТВ ЯДРА LINUX ДЛЯ ОБНАРУЖЕНИЯ АТАК DDOS И SYN-ФЛУД. — 2024.
5. DDoS атаки [Электронный ресурс]. — Режим доступа: https://en.wikipedia.org/wiki/Denial-of-service_attack (дата обращения: 29.10.2025).
6. Ботнет [Электронный ресурс]. — Режим доступа: <https://en.wikipedia.org/wiki/Botnet> (дата обращения: 29.10.2025).

ПРИЛОЖЕНИЕ А

Исходный код модуля

Листинг А.1 – Исходный код модуля

```
1 #include <uapi/linux/bpf.h>
2 #include <bpf_endian.h>
3 #include <bpf_helpers.h>
4 #include <uapi/linux/if_ether.h>
5 #include <uapi/linux/ip.h>
6 #include <linux/hash.h>
7
8 #define COOKIE_SECRET 0xdeadbeef
9
10 #define INTERNAL static __attribute__((always_inline))
11
12 #ifndef NDEBUG
13 #define LOG(fmt, ...) bpf_printk(fmt "\n", ##__VA_ARGS__)
14 #else
15 #define LOG(fmt, ...)
16 #endif
17
18 #ifndef memcpy
19 #define memcpy(dest, src, n) __builtin_memcpy((dest), (src),
    (n))
20 #endif
21
22 #ifndef memset
23 #define memset(dest, chr, n) __builtin_memset((dest), (chr),
    (n))
24 #endif
25
26 struct Packet {
27     struct xdp_md *ctx;
28     struct ethhdr *ether;
29     struct iphdr *ip;
30     struct tcphdr *tcp;
31 };
32
33 struct FourTuple {
34     __u32 saddr;
35     __u32 daddr;
```

```

36     __u16 sport;
37     __u16 dport;
38 };
39
40 INTERNAL __u32 hash_crc32(__u32 data, __u32 seed) {
41     return hash_32(seed ^ data, 32);
42 }
43
44 INTERNAL __u32 cookie_hash(struct FourTuple t) {
45     __u32 h = COOKIE_SECRET;
46     h = hash_crc32(t.saddr, h);
47     h = hash_crc32(t.daddr, h);
48     h = hash_crc32(((__u32)t.sport << 16) | t.dport, h);
49     return h;
50 }
51
52 INTERNAL __u32 cookie_make(struct FourTuple t, __u32
    client_seq) {
53     return client_seq + cookie_hash(t);
54 }
55
56 INTERNAL int cookie_check(struct FourTuple t, __u32 client_seq,
    __u32 ack_seq) {
57     return cookie_make(t, client_seq) == ack_seq;
58 }
59
60 #define MAX_CSUM_WORDS 32
61 #define MAX_CSUM_BYTES (MAX_CSUM_WORDS * 2)
62
63 INTERNAL __u32 sum16(const void *data, __u32 size, const void
    *data_end) {
64     __u32 s = 0;
65     #pragma unroll
66     for (__u32 i = 0; i < MAX_CSUM_WORDS; i++) {
67         if (2 * i >= size)
68             return s;
69         if (data + 2 * i + 2 > data_end)
70             return 0;
71         s += ((__u16 *)data)[i];
72     }
73     return s;

```



```

74 }
75
76 INTERNAL __u32 sum16_32(__u32 v) {
77     return (v >> 16) + (v & 0xffff);
78 }
79
80 INTERNAL __u16 carry(__u32 csum) {
81     csum = (csum & 0xffff) + (csum >> 16);
82     csum = (csum & 0xffff) + (csum >> 16);
83     return ~csum;
84 }
85
86 INTERNAL int process_tcp_syn(struct Packet *packet) {
87     struct xdp_md *ctx = packet->ctx;
88     struct ethhdr *ether = packet->ether;
89     struct iphdr *ip = packet->ip;
90     struct tcphdr *tcp = packet->tcp;
91     const void *data_end = (void *)ctx->data_end;
92
93     __u32 ip_len = ip->ihl * 4;
94     if ((void *)ip + ip_len > data_end)
95         return XDP_DROP;
96
97     __u32 tcp_len = tcp->doff * 4;
98     if ((void *)tcp + tcp_len > data_end)
99         return XDP_DROP;
100
101     struct FourTuple tuple = {
102         ip->saddr,
103         ip->daddr,
104         tcp->source,
105         tcp->dest
106     };
107
108     __u32 client_seq = bpf_ntohl(tcp->seq);
109     __u32 cookie = cookie_make(tuple, client_seq);
110
111     tcp->ack_seq = bpf_htonl(client_seq + 1);
112     tcp->seq = bpf_htonl(cookie);
113     tcp->ack = 1;
114

```

```

115     __u16 tp = tcp->source;
116     tcp->source = tcp->dest;
117     tcp->dest = tp;
118
119     __u32 ti = ip->saddr;
120     ip->saddr = ip->daddr;
121     ip->daddr = ti;
122
123     struct ethhdr tmp = *ether;
124     memcpy(ether->h_dest, tmp.h_source, ETH_ALLEN);
125     memcpy(ether->h_source, tmp.h_dest, ETH_ALLEN);
126
127     ip->check = 0;
128     ip->check = carry(sum16(ip, ip_len, data_end));
129
130     __u32 tcp_csum = 0;
131     tcp_csum += sum16_32(ip->saddr);
132     tcp_csum += sum16_32(ip->daddr);
133     tcp_csum += 0x0600;
134     tcp_csum += tcp_len << 8;
135     tcp->check = 0;
136     tcp_csum += sum16(tcp, tcp_len, data_end);
137     tcp->check = carry(tcp_csum);
138
139     LOG("SYN cookie sent %x", ip->daddr);
140     return XDP_TX;
141 }
142
143 INTERNAL int process_tcp_ack(struct Packet *packet) {
144     struct iphdr *ip = packet->ip;
145     struct tcphdr *tcp = packet->tcp;
146
147     struct FourTuple tuple = {
148         ip->saddr,
149         ip->daddr,
150         tcp->source,
151         tcp->dest
152     };
153
154     __u32 client_seq = bpf_ntohl(tcp->seq) - 1;
155     __u32 ack_seq = bpf_ntohl(tcp->ack_seq) - 1;

```

```

156
157     if (!cookie_check(tuple, client_seq, ack_seq)) {
158         LOG("COOKIE FAIL %x", ip->saddr);
159         return XDP_DROP;
160     }
161
162     LOG("COOKIE OK %x", ip->saddr);
163     return XDP_PASS;
164 }
165
166 INTERNAL int process_tcp(struct Packet *packet) {
167     struct tcphdr *tcp = packet->tcp;
168     __u16 flags = bpf_ntohs(tcp->flags) & (TH_SYN | TH_ACK);
169
170     if (flags == TH_SYN)
171         return process_tcp_syn(packet);
172     if (flags == TH_ACK)
173         return process_tcp_ack(packet);
174
175     return XDP_PASS;
176 }
177
178 INTERNAL int process_ip(struct Packet *packet) {
179     struct iphdr *ip = packet->ip;
180     if (ip->protocol != IPPROTO_TCP)
181         return XDP_PASS;
182
183     struct tcphdr *tcp = (struct tcphdr *) (ip + 1);
184     if ((void *) (tcp + 1) > (void *) packet->ctx->data_end)
185         return XDP_DROP;
186
187     packet->tcp = tcp;
188     return process_tcp(packet);
189 }
190
191 INTERNAL int process_ether(struct Packet *packet) {
192     struct ethhdr *ether = packet->ether;
193     if (ether->h_proto != bpf_htons(ETH_P_IP))
194         return XDP_PASS;
195
196     struct iphdr *ip = (struct iphdr *) (ether + 1);

```

```

197     if ((void *) (ip + 1) > (void *) packet->ctx->data_end)
198         return XDP_DROP;
199
200     packet->ip = ip;
201     return process_ip(packet);
202 }
203
204 SEC("prog")
205 int xdp_main(struct xdp_md *ctx) {
206     struct Packet packet = {};
207     packet.ctx = ctx;
208
209     struct ethhdr *ether = (struct ethhdr *) (void *) ctx->data;
210     if ((void *) (ether + 1) > (void *) ctx->data_end)
211         return XDP_PASS;
212
213     packet.ether = ether;
214     return process_ether(&packet);
215 }
216
217 char _license[] SEC("license") = "GPL";

```

ПРИЛОЖЕНИЕ Б

Файл сборки Makefile

Листинг Б.1 – Файл сборки

```
1 CLANG ?= clang
2 LLC ?= llc
3
4 KDIR ?= /lib/modules/$(shell uname -r)/build
5 ARCH ?= $(subst x86_64,x86,$(shell uname -m))
6
7 CFLAGS = \
8     -Ihelpers \
9     \
10    -I$(KDIR)/include \
11    -I$(KDIR)/include/uapi \
12    -I$(KDIR)/include/generated/uapi \
13    -I$(KDIR)/arch/$(ARCH)/include \
14    -I$(KDIR)/arch/$(ARCH)/include/generated \
15    -I$(KDIR)/arch/$(ARCH)/include/uapi \
16    -I$(KDIR)/arch/$(ARCH)/include/generated/uapi \
17    -D__KERNEL__ \
18    \
19    -Wno-int-to-void-pointer-cast \
20    \
21    -fno-stack-protector -O2 -g
22
23 xdp_%.o: xdp_%.c Makefile
24     $(CLANG) -c -emit-llvm $(CFLAGS) $< -o - | \
25     $(LLC) -march=bpf -filetype=obj -o $@
26
27 .PHONY: all clean
28
29 all: xdp_filter.o xdp_dummy.o
30
31 clean:
32     rm -f ./*.o
```

ПРИЛОЖЕНИЕ В

Исходный код скрипа для тестирования

Листинг В.1 – Исходный код скрипта для тестирования

```
1 #!/ bin / sh
2
3 [ $# -ge 1 ] || { echo >&2 "Usage: $0
    {up|down|status|attach|detach|log|exec|listen}"; exit 1; }
4 [ $UID -eq 0 ] || { echo >&2 "This script requires root."; exit
    1; }
5
6 action="$1"
7 netns=xdp-test
8 local=xdp-local
9 remote=xdp-remote
10 dummy=xdp_dummy.o
11 filter=xdp_filter.o
12
13 case "$action" in
14 up)
15     ip netns add ${netns}
16     ip link add ${remote} type veth peer name ${local}
17     ip link set ${remote} netns ${netns}
18     ip netns exec ${netns} ip address add 192.0.2.2/24 dev
        ${remote}
19     ip netns exec ${netns} ip link set ${remote} up
20     ip address add 192.0.2.1/24 dev ${local}
21     ip link set ${local} up
22     ;;
23
24 down)
25     ip netns delete ${netns}
26     ip link delete ${local}
27     ;;
28
29 status)
30     if ip netns list | grep ${netns} 2>&1 >/dev/null; then
31         echo "Stand is UP."
32     else
33         echo "Stand is DOWN."
34     fi
```

```

35     ;;
36
37 attach)
38     ip netns exec ${netns} ip -force link set dev ${remote} xdp
        object ${dummy} >/dev/null
39     ip -force link set dev ${local} xdp object ${filter} verbose
40     ;;
41
42 detach)
43     ip netns exec ${netns} ip link set dev ${remote} xdp off
44     ip link set dev ${local} xdp off
45     ;;
46
47 log)
48     echo -n 1 > /sys/kernel/debug/tracing/options/trace_printk
49     cat /sys/kernel/debug/tracing/trace_pipe
50     ;;
51
52 exec)
53     shift
54     ip netns exec ${netns} $@
55     ;;
56
57 listen)
58     ip netns exec ${netns} tcpdump -tnevi ${remote} not ip6
59     ;;
60 esac

```